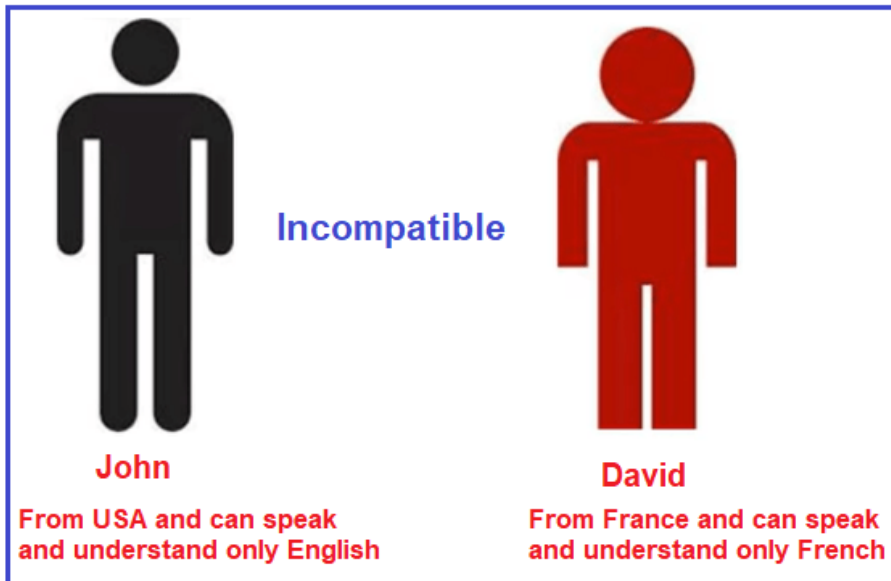


Adapter Design Pattern Real-time Example in C#

In this article, I am going to discuss the **Adapter Design Pattern Real-Time Example in C#**. Please read our previous article where we discussed the basic concepts of the [Adapter Design Pattern](#). Here, in this article, we will implement the Language Translator Example using the Adapter Design Pattern which is one of the best Real-Time examples of the Adapter Design Pattern.

Adapter Design Pattern Real-Time Example in C# – Language Translator

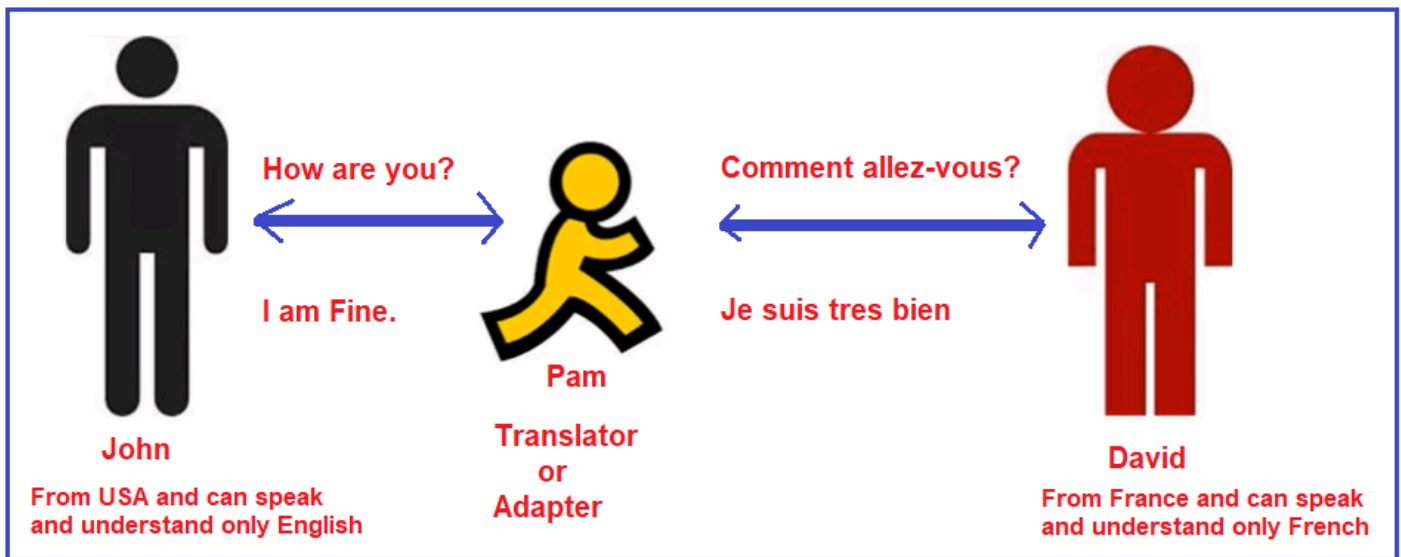
Let us understand the Language Translator Example in C# using Adapter Design Pattern. Please have a look at the following diagram for a better understanding. On the left-hand side, you can see a person called John and on the right-hand side, you can see a person called David.



The person John is from the USA and he can only speak and understand only English. On the other hand, person David is from France and he can only speak and understand French. Now, John wants to speak something to David and David wants to speak something back to John. But, currently, it is not possible because John only knows English and he can't understand French. In the same way, David knows only French and he can't understand English. So the above two interfaces are incompatible as they can't communicate.

How to Make Them Compatible?

In order to make them compatible with each other, what we need to do is, we need to introduce a Translator (Middleman) between John and David which is shown in the below image. Here, the person Pam acts as a Translator. Pam knows both English and French. She can speak and understand English, as well as translate English to French. She also speaks and understands French, as well as translates French into English. So, here, Pam is nothing but an Adapter.



Suppose John wants to ask **how are you** to David. Then what John will do is, he will ask how are you to Pam, and Pam will convert how are you to French and ask David. David will reply in French to Pam and Pam again converts that French to English and return to John. So, in this case, both John and David will communicate with each other. Here, the important thing is Pam and Pam is acting as an Adapter between John and David.

Implementation of Adapter Design Pattern Real-Time Example using C#:

Let us implement the above-discussed Language Translator Example step by step in C# using Adapter Design Pattern. As we already discussed in our previous article, the Adapter Design Pattern involves four components (Target, Client, Adaptee, and Adapter). Let us create these components by comparing them with our example.

Step1: Creating EnglishSpeaker Interface (Adaptee)

Create an interface with the name `IEnglishSpeaker` and then copy and paste the following code into it. This interface is having two methods i.e. `AskQuestion` and `AnswerFortheQuestion` and both these methods expect parameter words.

```
namespace AdapterDesignPatternRealTimeExample
{
    public interface IEnglishSpeaker
    {
        string AskQuestion(string Words);
        string AnswerFortheQuestion(string Words);
    }
}
```

In our example, John is a person who can speak and understand only English. So create a class file with the name **John.cs** and then copy and paste the following code into it. This John class implements the `IEnglishSpeaker` interface and provides implementations for the two abstract methods i.e. `AskQuestion` and `AnswerFortheQuestion`.

```
using System;
namespace AdapterDesignPatternRealTimeExample
{
    // John is from USA, So he can speak and understand only English
    public class John : IEnglishSpeaker
    {
        public string AskQuestion(string Words)
        {
            Console.WriteLine("Question Asked by John [English Speaker and  
Can understand only English] : " + Words);
        }
    }
}
```

```

        ITarget target = new Pam();
        string replyFromDavid =
target.TranslateAndTellToOtherPerson(Words, "French");
        return replyFromDavid;
    }

    public string AnswerFortheQuestion(string Words)
    {
        string reply = null;
        if (Words.Equals("where are you?",
StringComparison.InvariantCultureIgnoreCase))
        {
            reply = "I am in USA";
        }
        return reply;
    }
}
}
}

```

Step2: Creating FrenchSpeaker Interface (Adaptee)

Create an interface with the name IFrenchSpeaker and then copy and paste the following code into it. This interface is also having two methods (AskQuestion and AnswerFortheQuestion) and again these two methods expect parameter words.

```

namespace AdapterDesignPatternRealTimeExample
{
    public interface IFrenchSpeaker
    {
        string AskQuestion(string Words);
        string AnswerFortheQuestion(string Words);
    }
}

```

In our example, David is the person who can only speak and understand only French. So, create a class with the name David and then copy and paste the following code into it. This class is going to implement the IFrenchSpeaker interface and provide implementations for the AskQuestion and AnswerFortheQuestion abstract methods.

```

using System;
namespace AdapterDesignPatternRealTimeExample
{
    // David is from France and can speak and understand only French
    public class David : IFrenchSpeaker
    {
        public string AskQuestion(string Words)
        {
            Console.WriteLine("Question Asked by David [French Speaker and
Can understand only French] : " + Words);
            ITarget target = new Pam();
            string replyFromJohn =
target.TranslateAndTellToOtherPerson(Words, "English");
            return replyFromJohn;
        }
        public string AnswerFortheQuestion(string Words)
        {

```

```

        string reply = null;
        if (Words.Equals("comment allez-vous?",
StringComparison.InvariantCultureIgnoreCase))
        {
            reply = "Je suis très bien";
        }
        return reply;
    }
}

```

Step3: Creating Target interface

Create an interface with the name ITarget and then copy and paste the following code into it. The ITarget interface defines the abstract TranslateAndTellToOtherPerson method which is going to be implemented by the Adapter.

```

namespace AdapterDesignPatternRealTimeExample
{
    public interface ITarget
    {
        string TranslateAndTellToOtherPerson(string words, string
convertToWhichLanguage);
    }
}

```

Step4: Creating Pam (Translator or Adapter)

Create a class with the name Pam and then copy and paste the following code into it. The following Pam class implements the ITarget interface and provides implementations for the TranslateAndTellToOtherPerson method. What this TranslateAndTellToOtherPerson method does is, it will translate the word to English and talk to John and again it also translates the word to French and talks to David. This method acts as a two-way communicator (it can communicate with both persons).

This class is also having two methods. One method is used to convert English to French and the other method is used to convert French to English.

```

using System;
using System.Collections.Generic;
namespace AdapterDesignPatternRealTimeExample
{
    // Adapter or Translator
    // Pam can speak and understand both English and French
    // She acts as an Adapter or Translator
    public class Pam:ITarget
    {
        static Dictionary < string, string > EnglishFrenchDictionary = new
Dictionary < string, string > ();
        static Dictionary < string, string > FrenchEnglishDictionary = new
Dictionary < string, string > ();
        David david = new David ();
        John john = new John ();

        static Pam ()
        {

```

```

        EnglishFrenchDictionary.Add ("how are you?", "comment allez-
vous?");
        EnglishFrenchDictionary.Add ("I am in USA", "Je suis aux Etats-
Unis");
        FrenchEnglishDictionary.Add ("Je suis trC(s bien", "I am
fine");
        FrenchEnglishDictionary.Add ("oC9 C*tes-vous?", "where are
you?");
    }
    public string TranslateAndTellToOtherPerson (string Words, string
ConvertToWhichLanguage)
    {
        if (ConvertToWhichLanguage.Equals ("English",
StringComparison.InvariantCultureIgnoreCase))
        {
            string EnglishWords = ConvertToEnglish (Words);
            Console.WriteLine ("\nPam Converted \"" + Words + " \" to \""
+ EnglishWords + " and send the question to John");
            string EnglishWordsReply = john.AnswerFortheQuestion
(EnglishWords);
            Console.WriteLine ("Pam Got reply from John in English : " +
 "\"" + EnglishWordsReply + "\"");
            string FrenchConverted = ConvertToFrench (EnglishWordsReply);
            Console.WriteLine ("Pam Converted " + "\"" + EnglishWordsReply
+ "\"" + " to " + "\"" + FrenchConverted + "\"" + " and send back to
David\n");
            return FrenchConverted;
        }
        else if (ConvertToWhichLanguage.Equals("French",
StringComparison.InvariantCultureIgnoreCase))
        {
            string FrenchWords = ConvertToFrench (Words);
            Console.WriteLine ("\nPam Converted \"" + Words + " \" to \""
+ FrenchWords + " and send the question to David");
            string FrenchWordsReply = david.AnswerFortheQuestion
(FrenchWords);
            Console.WriteLine ("Pam Got reply from David in French : " +
 "\"" +FrenchWordsReply + "\"");
            string EnglishConverted = ConvertToEnglish (FrenchWordsReply);
            Console.WriteLine ("Pam Converted " + "\"" + FrenchWordsReply
+ "\"" + " to " + "\"" + EnglishConverted + "\"" + " and send back to
John\n");
            return EnglishConverted;
        }
        else
        {
            return "Sorry Cannot Covert";
        }
    }
    public string ConvertToFrench (string Words)
    {
        return EnglishFrenchDictionary[Words];
    }
    public string ConvertToEnglish (string Words)

```

```

        {
            return FrenchEnglishDictionary[Words];
        }
    }
}

```

Note: This class acts as an Adapter as it is adapting something before communicating.

Step5: Client

Our main method is going to be the client for our application. So, please modify the Main method as shown below.

```

using System;
namespace AdapterDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            string replyFromDavid = new John().AskQuestion("how are you?");
            Console.WriteLine("Reply From David [French Speaker can Speak and Understand only French] : " + replyFromDavid);
            Console.WriteLine();
            string replyFromJohn = new David().AskQuestion("où êtes-vous?");
            Console.WriteLine("Reply From John [English Speaker can Speak and Understand only English] : " + replyFromJohn);

            Console.Read();
        }
    }
}

```

Output:

```

Question Asked by John [English Speaker and Can understand only English] : how are you?
Pam Converted "how are you? " to "comment allez-vous? and send the question to David
Pam Got reply from David in French : "Je suis très bien"
Pam Converted "Je suis très bien" to "I am fine" and send back to John

Reply From David [French Speaker can Speak and Understand only French] : I am fine

Question Asked by David [French Speaker and Can understand only French] : où êtes-vous?
Pam Converted "où êtes-vous? " to "where are you? and send the question to John
Pam Got reply from John in English : "I am in USA"
Pam Converted "I am in USA" to "Je suis aux Etats-Unis" and send back to David

Reply From John [English Speaker can Speak and Understand only English] : Je suis aux Etats-Unis

```

In the next article, I am going to discuss the [Facade Design Pattern in C#](#) with Real-Time Examples. Here, in this article, I try to explain one of the best **Adapter Design Patterns Real-time** Examples in C#. I hope now you understood the need and use of the Adapter Design Pattern in Real-time Applications in C#.